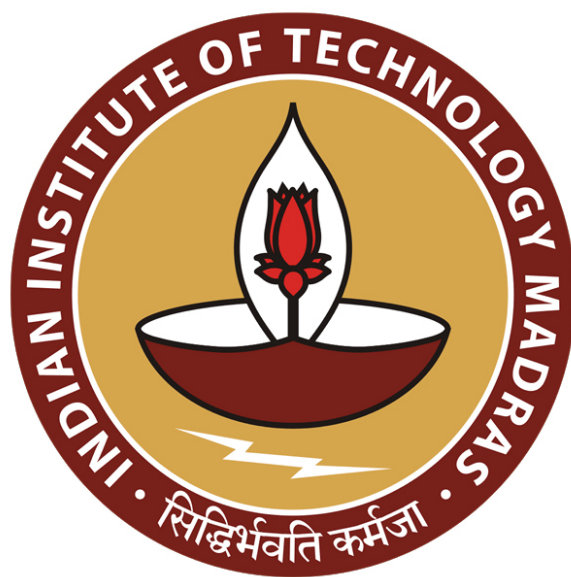




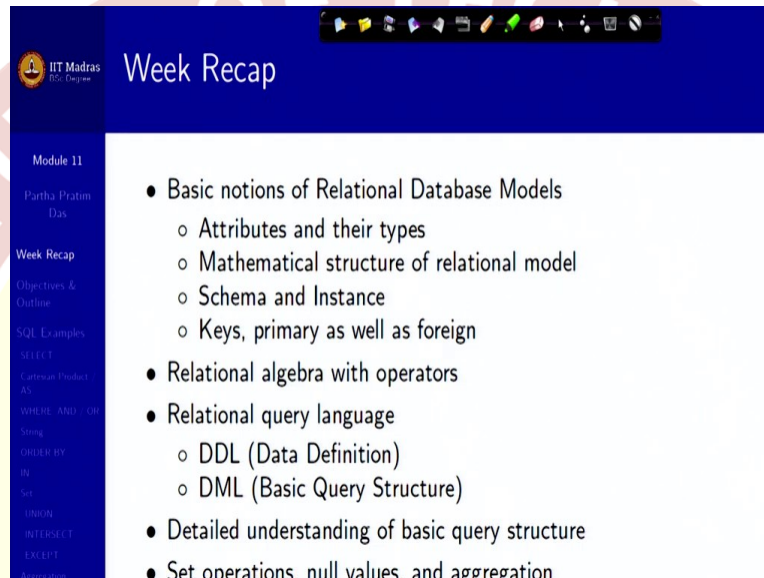
IIT Madras
ONLINE DEGREE

Database Management Systems

Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
SQL Examples

Welcome to week 3, module 11 of Database Management Systems course in IIT Madras online B.Sc program.

(Refer Slide Time: 0:27)



IIT Madras
BSc Degree

Week Recap

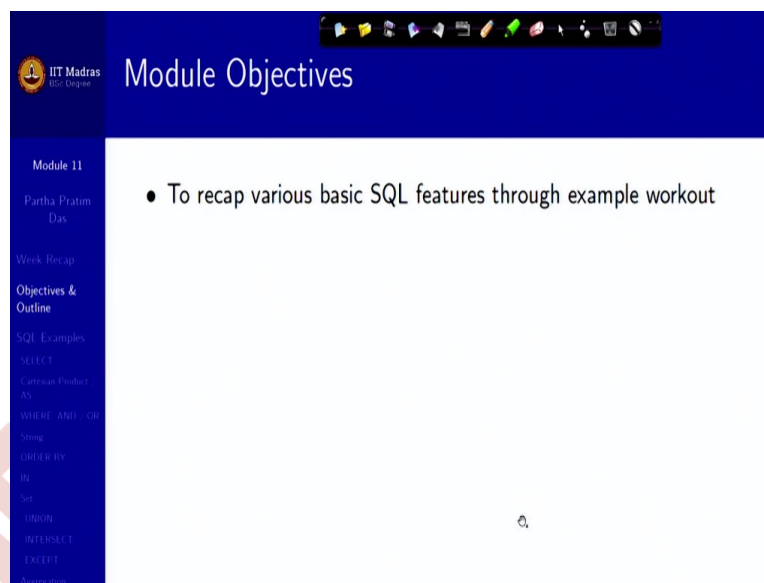
Module 11
Partha Pratim Das

- Basic notions of Relational Database Models
 - Attributes and their types
 - Mathematical structure of relational model
 - Schema and Instance
 - Keys, primary as well as foreign
- Relational algebra with operators
- Relational query language
 - DDL (Data Definition)
 - DML (Basic Query Structure)
- Detailed understanding of basic query structure
- Set operations, null values, and aggregation

In the last week, over all the 5 modules, we have done a detailed introduction to the fundamentals of relational database models, attributes, types. The basic mathematical structure we will have to do deeper later on. Then, talking about schema, instance, different types of keys and the basic operations like select, project, join, these kind of basic operations of relational algebra.

And introduced the basic structure of queries, both in terms of the data definition language as well as for the data manipulation language. And using several features of the SQL language select from where and then, different extensional features on that, we have shown how to form different types of queries, relatively simple ones I would say and we have also shown the use of different set operations, use of null values, aggregation functions and so on. So, it has been a quite an active week last week talking about relational database model and SQL.

(Refer Slide Time: 1:52)



IIT Madras
For Original

Module Objectives

Module 11
Partha Pratim Das

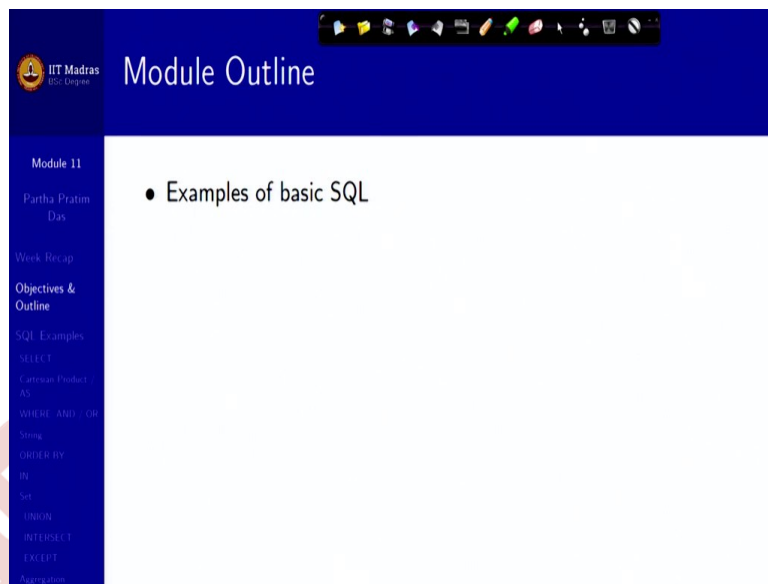
- To recap various basic SQL features through example workout

Week Recap
Objectives & Outline
SQL Examples
SELECT
Cartesian Product
AS
WHERE AND OR
Group
ORDER BY
IN
Set
UNION
INTERSECT
EXCEPT
Aggregates

So, we start this week today by doing a quick recap of what we have done. This recap is not in terms of restating what we have learned last week, let us try trying to solve an example each for most of the major features that we have discussed, because it is very important for you now, to actually start practicing coding in SQL, because it is as you have seen, it is a kind of a programming language, but it is a different style of programming language very different from all that you have done so far which has significantly been procedural in nature.

But this is the first declarative language you are doing and based on a very different kind of model. So, unless you practice a lot, you will not be able to internalize the mechanisms to extract queries, making changes to the database structure or to the manipulation of tuples and so on, and going forward, it will become very difficult. So, we will be doing a quick roundup of the, of some key examples here.

(Refer Slide Time: 3:13)

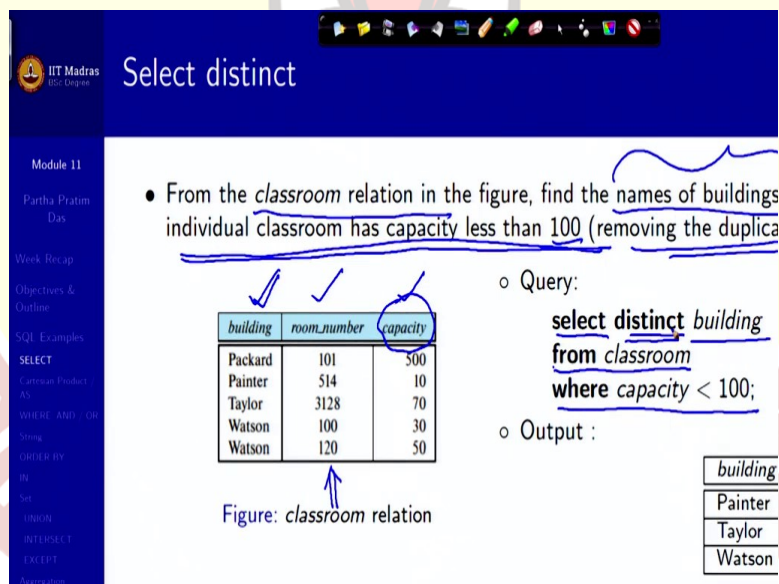


Module Outline

- Examples of basic SQL

We will continue to use the same database schema of the university database that we had used last week. So, you could anytime refer to that.

(Refer Slide Time: 3:23)



Select distinct

- From the classroom relation in the figure, find the names of buildings individual classroom has capacity less than 100 (removing the duplicates)

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Figure: classroom relation

Query:

```
select distinct building
from classroom
where capacity < 100;
```

Output :

building
Painter
Taylor
Watson



IIT Madras
BSc Degree

Module 11

Partha Pratim Das

Week Recap

Objectives & Outline

SQL Examples

SELECT

Cartesian Product

AS

WHERE AND OR

String

ORDER BY

IN

Set

UNION

INTERSECT

EXCEPT

Aggregation

Select distinct

- From the *classroom* relation in the figure, find the names of buildings individual classroom has capacity less than 100 (removing the duplicates)

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Query:

```
select distinct building
from classroom
where capacity < 100;
```

Output :

building
Painter
Taylor
Watson

Figure: *classroom* relation



IIT Madras
BSc Degree

Module 11

Partha Pratim Das

Week Recap

Objectives & Outline

SQL Examples

SELECT

Cartesian Product

AS

WHERE AND OR

String

ORDER BY

IN

Set

UNION

INTERSECT

EXCEPT

Aggregation

Select all

- From the *classroom* relation in the figure, find the names of buildings individual classroom has capacity less than 100 (without removing the duplicates)

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Query:

```
select all building
from classroom
where capacity < 100;
```

Output:

building
Painter
Taylor
Watson
Watson

Figure: *classroom* relation

So, the first is the basic select that is you are given a relation and you have to find out information from that. So, from the classroom relation, we want to find the names of buildings in which individual classroom has to be taken out, which has a capacity less than 100. So, because it is classroom relation, you immediately know the 'from' part of the query. So, 'from' clause will be from classroom that says that we are talking about we will be dealing with the classroom relation.

Then, we are saying that individual classroom capacity must be less than 100. So, in the relation there are 3 attributes, room number, capacity and building obviously, it goes without saying though, we have not looked at the schema, we can very easily understand by common sense that room number is the key here because suddenly there will be multiple classrooms in a building and two or more classrooms can always have the same capacity.

So, it is a capacity which is bounded by 100. So, in the where clause condition, which is the predicate, we set capacity is less than 100. So, that tells us that we will get classrooms only where the capacity is less than 100. And then finally, we put the 'select' clause which says that what do you want, you want names of the buildings. So, we want the name of the building, so we say select building.

And as you have learnt that 'select from where' this by default gives you a multiset so duplicates are not removed. So, here we specifically want remove duplicates. So, we are specifying the distinct qualifier. So, select distinct, building I will remove these markings now.

So, select distinct building from classroom where capacity is less than 100. So, it goes to the table looks at capacity less than 100, this will not qualify, this will qualify, this will qualify, this will qualify, this will qualify. So, this condition is satisfied. So, in terms of distinct build in terms of buildings, this is what do we will get; Painter, Taylor, Watson, Watson, naturally we want to remove duplicate you are using distinct, so one of the Watson's have to go and this is what is your output.

So, output is a table having only one column that is corresponding to building and it has 3 rows. So, this is the commonest, simplest kind of select from where that you can have and this by itself covers a huge part of the types of queries that you write. Naturally, if we look at, if we want that we would not remove duplicate this query is same as before, except that I am sorry, except that we are saying without removing duplicate.

So, when you do not want to remove duplicates, naturally you do not say 'distinct' you can say 'all'. So, which will mean that again these four are selected by the where clause, this is my relation taking a projection on building because I want select building and the Watson repeated will not be removed in this case. So, you have two instances of the Watson.

Now, it may be noted that again, I am saying it repeatedly so that it gets into your mind that select from where gives you a multiset. So, even if you do not say all, by default, it will give you this result. So, giving all is just kind of explicitly saying okay give me everything, but it is optional, if you do not give anything, then it will give you the multiset, it will give you the repetitions, it will not remove the repetitions. For repetitions to be removed, you need to put the distinct. So, this was the simplest of queries that you could have. And you can keep tracking on the left as to where we are progressing.

(Refer Slide Time: 8:01)

Cartesian Product

- Find the list of all students of departments which have a budget < \$0.1million

```
select name, budget
from student, department
where student.dept_name = department.dept_name and
budget < 100000;
```

- The above query first generates every possible student-department pair, which is the Cartesian product of student and department. Then, it filters all the rows with `student.dept_name = department.dept_name` and `budget < 100000`.
- The common attribute `dept_name` in the resulting table are

name
Brandt
Peltier
Levy
Sanchez
Snow
Aoi
Bourikas
Tanaka

Cartesian Product

- Find the list of all students of departments which have a budget < \$0.1million

```
select name, budget
from student, department
where student.dept_name = department.dept_name and
budget < 100000;
```

- The above query first generates every possible student-department pair, which is the Cartesian product of student and department. Then, it filters all the rows with `student.dept_name = department.dept_name` and `budget < 100000`.
- The common attribute `dept_name` in the resulting table are renamed using the relation name - `student.dept_name` and `department.dept_name`)

name	budget
Brandt	50000.00
Peltier	70000.00
Levy	70000.00
Sanchez	80000.00
Snow	70000.00
Aoi	85000.00
Bourikas	85000.00
Tanaka	90000.00

Next what we say is a Cartesian product. Cartesian product is a, so select from where actually covers both your projection as well as selection, these two operators of the relational algebra. Now, Cartesian product is where you have two or more relations. So, let us look at we are having two relations, student and department and we want to take a Cartesian product that is make all combination of tuples between students and department.

So, if we just, if we did not write this where part, if we just did select name, budget from student to department, it will give you all combinations of students and department. But we have qualified that by saying that the student.department is equal to department.department. Now the student.dept_name is equal to department.dept_name.

So, what it will retain only those tuples, only those rows where the dept_name under the, from the student relation and the dept_name from the department relation are same designating that this student actually studies in this department. So, that is that so it is a Cartesian product with a qualifier which we will soon see is what will turn out to be I mean we can do this very directly by using something called join which we will introduce in the later module.

And further we are saying that, we say that list of students of all departments so this is what comes here. And we want a budget which is less than \$0.1million which is \$100,000. So, the budget is less than so it will have to match and the budget will have to be less. So, if you check back on the two relations that we have already given you in the example, then you will see that these are the tuples which will survive through it.

Where we finally show the name of the student we are not showing, by projection we are not showing the name of the department, we are showing the name of the student as required and the budget as required that the budget has to be more than your 10000 or whatever less than \$10,000 in every case you can check that is the case. So, this is the little bit extended query where instead of using 1 relation, now, we are using 2 relations.

(Refer Slide Time: 11:00)

Rename AS Operation

- The same query in the previous slide can be framed by renaming the tables as shown below.

```

select S.name as studentname, budget as deptbudget
from student as S, department as D
where S.dept_name = D.dept_name and budget < 100000;
  
```

Handwritten annotations on the query:

- Blue arrow pointing from 'S' to 'studentname'.
- Blue arrow pointing from 'D' to 'deptbudget'.
- Blue arrow pointing from 'S.dept_name' to 'D.dept_name'.
- Blue arrow pointing from 'budget' to 'deptbudget'.

- The above query renames the relation *student* as *S* and the relation *department* as *D*
- It also displays the attribute *name* as *StudentName* and *budget* as *DeptBudget*.
- Note that the budget attribute does not have any prefix

studentname
Brandt
Peltier
Levy
Sanchez
Snow
Aoi
Bourikas
Tanaka

Now, let us go further, the another way to write this relation is using the renaming feature, that is the 'as' we can any attribute or any relation I can rename with a, to a different name. So, first if you look at the relations the student I have renamed as S, department as I have renamed as D and writing this 'as' is optional also you can just say it student S comma

department D, it will mean the same thing but writing as I prefer to be not smart, but explicit that way chances of error gets less.

So, now I write S.dept_name equal to D.dept_name, we do not need to write the student or department that makes it a little bit less of typing. And also on the select clause we are saying the S.name which is actually student.name because of this alias is given a new, this attribute is given a new name student name, budget is given a new name, dept budget, you can see that I could have written this as D.budget also, because D is a department and department has a budget.

But I am not writing that, that is I mean I can write there is nothing wrong, but I do not need to write it because the budget attribute occurs only in the department. It does not occur in the students. So, I do not need to distinguish whereas, the department names occurs in both. So, I need to distinguish which department name it is the department name of the student or is it the department name of the department table that I am talking of.

So, these are the variabilities. So, instead of budget here I could have written D.budget as well. So, this is just another way of writing that query when we get more and more complex, you will probably or when you do want to do a self-cross product cross product between a relation and itself, you will certainly need to use rename as you have seen already in the earlier discussion.

(Refer Slide Time: 13:16)

Select: AND and OR

- From the *instructor* and *department* relations in the figure, find out the names of department is Finance or whose department is in any of the following buildings: Watson

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califleri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

department

dept_name	building	budget
Biology	Watson	90000

Query:

```
select name
from instructor I, department D
where D.dept_name = I.dept_name
and (I.dept_name = 'Finance'
or building in ('Watson'))
```

Output:

name
Srinivasan
Wu
Einstein
Gold

IIT Madras
DSC Database

Select: AND and OR

Module 11
Partha Pratim Das
Week Recap
Objectives & Outline
SQL Examples
SELECT
Correlation Functions
Agg
WHERE AND / OR
String
ORDER BY
IN
Set
UNION
INTERSECT
EXCEPT
Miscellaneous

- From the *instructor* and *department* relations in the figure, find out the names of department is Finance or whose department is in any of the following buildings: V

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

department

dept_name	building	budget
Biology	Watson	90000

Query:

```
select name
from instructor I, department D
where D.dept_name = I.dept_name
and (I.dept_name = 'Finance'
or building in ('Watson', 'Taylor'))
```

Output:

name
Srinivasan
Wu
Einstein
Gold
Katz

Certainly, in the where clause, you have a predicate. So, the predicate can use any operator of the Boolean algebra AND, OR, NOT. So, we are trying to see that we want to find names of all instructors whose department is finance or whose department is in Watson or Taylor. So, name, we want names of all instructors. So, instructor table will have to be there, instructor table will be required, we want that the name or the department's building should be either Watson or Taylor and that will satisfy. So, the department table is also required.

So, this is how you decide that in the from, on the from clause you have two relations instructor and department. Now, what is the condition that you need? You need to know the department of a faculty, department of an instructor because whose department, whose department. So, you need to make sure that the department of the faculty of the instructor and the department name in the department has to be equal exactly the similar thing we are doing for students there.

So, first part of the where clause has D.dept_name is equal to I.dept_name after renaming. Then we say that what is the condition on which we choose? This is a base condition this will make sure that the instructor and department are properly correlated. That is in other words, the join is happening and then what you have, you say that either the department of the instructor is finance. So, I.dept_name is equal to finance or building should be Watson or Taylor.

So, we are using the 'in' clause here by doing in Watson or Taylor. So, it means that if the value of building is Watson or the value of building is Taylor, this will be true, this predicate will be true, if it is something other than that, this predicate would be false. When did this

using this in we are making it simpler because otherwise I would have to again do or building equal to Watson or building equal to Taylor lot more of verbosity as well as chances of error.

So, what if you just carefully look at how it is composed, you can see that this is this join condition has to happen irrespective of anything else that is the validity of the query and rest of it is the total condition. So, you have 'and' and look at this parenthesis, which makes sure that this joining condition will have to be true, as well as this condition has to be true for the tuple to be selected.

So, that is how you have and here, but within the selection criteria of the tuple, you have either the instructor is from finance or his or her building is Watson or Taylor. So, you have 'or' in between them. If you do not put this parenthesis, it will be wrong. Because this will become and, or and something, you will have to then figure out which order they should be evaluated and so on. So, when you have multiple such connectives and, or, or qualifiers as not, you should be it is better to put the correct set of parentheses according to the semantics. So, that is how this can be done.

(Refer Slide Time: 17:08)

String Operations

- From the *course* relation in the figure, find the titles of all courses with three alphabets indicating the department.

Query:

```
select title
from course
where course_id like '___%';
```

Output:

title
Intro. to Biology
Genetics
Computational Biology
Investment Banking
World History
Physical Principles

course_id	title	deptname	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure: course relation

Other we showed is the use of string which can be used to match a variety of types of values. Here we have the table of courses, and what we are trying to show is how to pick up the find the titles of all courses, whose course_id has three alphabets indicating the departments, you can see some have 3, some have 2, and then there is a hyphen and then the number. So, how do I say that there has to be 3 alphabets?

So, I want 3 characters to come before the hyphen. So, I put 3 dashes, underscores, and then the hyphen, and then anything else is allowed. So, I put a wild string percentage, which will match everything. So, this will get matched, this will get matched, this will get matched. But this will not get matched, because CS has 2 characters followed by a hyphen.

So, in this way, if you check these all these a whole of CS, EE, MU, these will get excluded. And whatever remains, then I am doing a projection on title. So, I will get the titles of those courses. So, this is how a variety of ways you can use strings and it is very powerful in this way.

(Refer Slide Time: 18:31)

Order By

- From the *student* relation in the figure, obtain the list of all students in alphabetic order and within each department, in decreasing order of total credits.

Query:

```
select name, dept_name, tot_cred
from student
order by dept_name ASC, tot_cred DESC
```

Output:

name	dept_name
Tanaka	Biology
Zhang	Comp. Sci.
Brown	Comp. Sci.
Williams	Comp. Sci.
Shankar	Comp. Sci.
Bourikas	Elec. Eng.
Aoi	Elec. Eng.
Chavez	Finance
Brandt	History

Figure: student relation

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

The list is first sorted in alphabetic order

I think this was easy that how do you order your output all that I am trying to show here, order by basically will say whatever attribute you have given the result tuples will be ordered by that by default it is ascending, but if you want you can put 'ASC' it is optional, but if you want descending you will have to put descending what I am showing here is ordering by two tuples.

So, what do they do is department name ASC see that is it is to be within alphabetic order of departments and within each department decreasing order of total credits. So, then total credit decreasing. So, if you see in computer science, first of all you have biology, computer science, this is the lexicographical order and in computer science, you have a decreasing order of total credits to be compiled here together.

So, it is within computer science this is if I change this order, if I put this first and this next, then it will first get sorted by this total credit and within equal value of total credit it will get

sorted by the name of the department. So, this the in order by it is very important to know, to decide which is the ordering that you actually want of the attributes.

(Refer Slide Time: 20:01)

In Operator

- From the *teaches* relation in the figure, find the IDs of all courses taught in Fall of 2018.

o Query:

```
select course_id
from teaches
where semester in ('Fall')
and year=2018;
```

o Output:

course_id
CS-315
FIN-201
MU-199
HIS-351

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure: teaches relation

'in' is we have already discussed. So, the purpose of 'in' is I think clear. So, I will skip this you just check it out at home.

(Refer Slide Time: 20:11)

Set Operations: union

- For the same question in the previous slide, we can find the solution using the union operator as follows.

o Query:

```
select course_id
from teaches
where semester
in ('Fall', 'Spring')
and year=2018;

union

select course_id
from teaches
where semester
in ('Fall', 'Spring')
and year=2018;
```

o Output:

course_id
CS-315
FIN-201
MU-199
HIS-351

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure: teaches relation

In set we have three operations: union, intersection and difference which is except here. So, we say for the, what we are trying to find out is we want to find the solution using union. So, we say this is the courses that are offered in 2018. So, semester is fall, this is courses offered in spring and then you take them together, then you get the total set of courses and please recollect that union is a pure set operation, it is not a multiset operation unlike select.

So, it will remove all duplicate, if you do not, if you want the duplicates to be retained, that is some courses which are offered in both should come twice, then you will have to write union all.

(Refer Slide Time: 21:04)

Set Operations (2): intersect

- From the *instructor* relation in the figure, find the names of all instructors in either the Computer Science department or the Finance department whose salary is < 80000.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Callieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Query:

```
select name
from instructor
where dept_name in ('Comp. Sci.', 'Finance')
intersect
select name
from instructor
where salary < 80000;
```

Output:

name
Srinivasan

Figure: instructor relation

Similarly, you have intersect here, where you say the names of all instructors who taught either in computer science department or in finance department and whose salary is this. So, you have first this either or you put it in terms of 'in' so that any of the values will be fine get the instructors get all instructors whose salary is less than 80,000. So, it is kind of a 'and' condition that is happening here.

So, you subtract, you do the intersection, not subtract you do the intersection that is those who are common that is who are in computer science or finance and as a salary less than 80,000 will come in the intersection and that is the result that you will see if you check with the table. I will naturally urge you to check with all the results that we have given here.

(Refer Slide Time: 22:05)

Set Operations (3): except

- From the *instructor* relation in the figure, find the names of all instructors in either the Computer Science department or the Finance department whose salary is either ≥ 90000 or ≤ 70000 .

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Callieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure: *instructor* relation

Query:

```
select name
from instructor
where dept_name in
except
select name
from instructor
where salary < 90000
```

Output:

name
Wu
Brandt

Note that the same can be achieved using the

Similarly, you are saying find names of all instructors who taught either in computer science or in finance this part remains same and whose salary is either less than 90,000 greater than 90,000 or less than 70,000. So, what we are doing here is we are showing this by except. So, we have except so, we get all the faculty who qualify for the list and then all the faculty who satisfy the salary criteria, we just take them out.

You can, you could have obviously done it using a more complex where clause also, but possibly it is easier to understand it through the set theoretic operations if there is such a clear notion of two sets or more sets getting formed.

(Refer Slide Time: 22:59)

Aggregate functions: avg

- From the *classroom* relation given in the figure, find the names and total capacity of each building whose average capacity is greater than 25.

building	room_number	capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Figure: *classroom* relation

Query:

```
select building,
from classroom
group by building
having avg (cap
```

Output:

building
Taylor
Packard
Watson

Finally, aggregation is easy. So, all that you will have to remember that for aggregation we will need to group by on the entity on whose property we will aggregate here that entity is building from the classroom find the names and average capacity of each building, average capacity of each building this is what I want. So, classrooms comes clearly you want to find the name and average. So, this comes clearly.

Since I want it for each building, you will group by building so together take on the building and take the capacity and we are again done some qualify is just not grouping, but we will consider only those capacity, only those who whose average capacity is greater than 25. So, you take the buildings, compute the average if the average is greater than 25, then only this becomes true and then it comes in the output otherwise, it does not come in the output.

So, this is how in this case as it turns out, you have, you will not have Painter in the output. Clearly because Painter is just one instance. So, its average is that instance itself which is 10 which is not greater than 25. So, Painter is not there in the output.

(Refer Slide Time: 24:33)

Aggregate functions (2): min

- From the *instructor* relation given in the figure, find the least salary of an instructor among all the instructors.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Query:
`select min(salary)
from instructor;`

Output:
least
40000

Figure: *instructor* relation



IIT Madras
BSc Degree

Module 11

Partha Pratim Das

Week Recap

Objectives & Outline

SQL Examples

SELECT

Cartesian Product

AS

WHERE AND OR

String

ORDER BY

IN

Set

UNION

INTERSECT

EXCEPT

Aggregates

Aggregate functions (3): max

- From the *student* relation given in the figure, find the maximum cred any student among all the students.

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Ianaka	Biology	120

o Query:

```
select max(tot_cred) as
from student;
```

o Output:

max_cred
120

Similarly, you can do minimum, which is very very similar doing the minimum finding the minimum salary, you can do maximum finding the maximum credit. Just work this out with the given table instance and convince yourself that they work properly.

(Refer Slide Time: 24:53)



IIT Madras
BSc Degree

Module 11

Partha Pratim Das

Week Recap

Objectives & Outline

SQL Examples

SELECT

Cartesian Product

AS

WHERE AND OR

String

ORDER BY

IN

Set

UNION

INTERSECT

EXCEPT

Aggregates

Aggregate functions (4): count

- From the *section* relation given in the figure, find the number of courses in each building.

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

o Query:

```
select building,
count(course_id)
from section
group by building;
```

o Output:

building
Taylor
Packard
Painter

And finally, you have count which counts the number of instances, number of rows basically. So, what you are saying that from the section relation, so, this is the section relation given in the figure, find the number of courses run in each building. It is run in each building from the section relation. So, from section run in each building each building means I have to group by the building, and then what do I want to find out, I want to find out how many courses are run.

So, I want to count, I will count by course_id naturally, the building is grouped. So, it is grouped according to building like say, Taylor here, Taylor here. So, these are grouped together, and then I will count the number of courses that will be there. So, there are, as you can see, there are 5 such courses will come, so where your count id is 5.

So, I am counting and then you can see that there are duplications of course_ids, and they will be counted as distinct. Because, naturally, I am just counting the available number of rows in this.

(Refer Slide Time: 26:31)

Aggregate functions (5): sum

- From the *course* relation given in the figure, find the total credits offered by each department.

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Query:

```
select dept_name,
       sum(credits) as
from course
group by dept_name
```

Output:

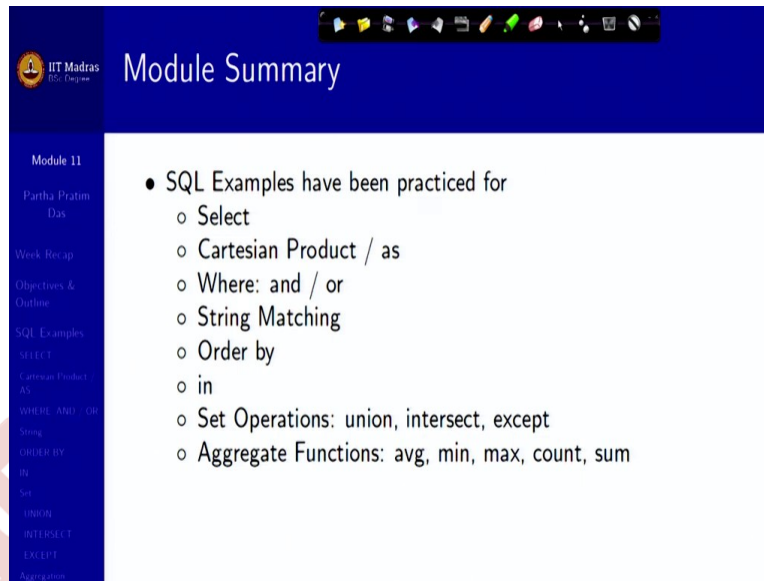
dept_name
Finance
History
Physics
Music

Handwritten calculation: $4+4+3=11$ (for Biology)

Sum is what you use to find out the total. So, for example, here, we are finding out the total credit offered by each state, each department. So, it is from the course table, so from course, offered by each department, each department, so group by is department name, and what we are finding the sum of is total credit. So, we sum under credit to get that. So if we have, say, if I consider, say music, I will get 3, only one entry. If I consider biology, there are 3 entries 4+4+3. So, this will be 11. Biology, biology is 11.

Now, these are offering in an haphazard order, if I want, I can, again, put an order by on this on the department name and get it in the order of biology, computer science like that. Those are all different choices.

(Refer Slide Time: 27:39)



The screenshot shows a presentation slide titled "Module Summary" from IIT Madras. The slide is divided into a left sidebar and a main content area. The sidebar lists the following items: Module 11, Partha Pratim Das, Week Recap, Objectives & Outline, SQL Examples, SELECT, Cartesian Product, AS, WHERE AND OR, String, ORDER BY, IN, Set, UNION, INTERSECT, EXCEPT, and Aggregate Functions. The main content area contains a bulleted list of SQL topics practiced for:

- SQL Examples have been practiced for
 - Select
 - Cartesian Product / as
 - Where: and / or
 - String Matching
 - Order by
 - in
 - Set Operations: union, intersect, except
 - Aggregate Functions: avg, min, max, count, sum

So, this was just a quick roundup of the most common and most widely used, this is kind of the foundational part of the SQL language and it is used. And I would again, again, reiterate and urge that you practice a lot of examples, there will be tutorials on this, there will be practice problems also given to you.

But get confident on this. And as we move forward through this week, you will get to see lot more of SQL features which are more advanced and by which you can do more interesting things. So, thank you very much for your attention and let us meet and discuss more of SQL in the next module.